

SWO - Sistema de Gestión de Incidencias (Service Desk)

SENA - Tecnología en Análisis y Desarrollo de Software
Proyecto Formativo — Ficha de Caracterización del Proyecto

Tabla de Contenidos

- [Descripción del Proyecto](#)
 - [Arquitectura](#)
 - [Módulos del Sistema](#)
 - [Stack Tecnológico](#)
 - [Requisitos Previos](#)
 - [Instalación y Ejecución](#)
 - [Variables de Entorno](#)
 - [API REST — Endpoints](#)
 - [Pruebas](#)
 - [Despliegue](#)
 - [Control de Versiones](#)
 - [Estructura del Proyecto](#)
 - [Documentación Adicional](#)
-

Descripción del Proyecto

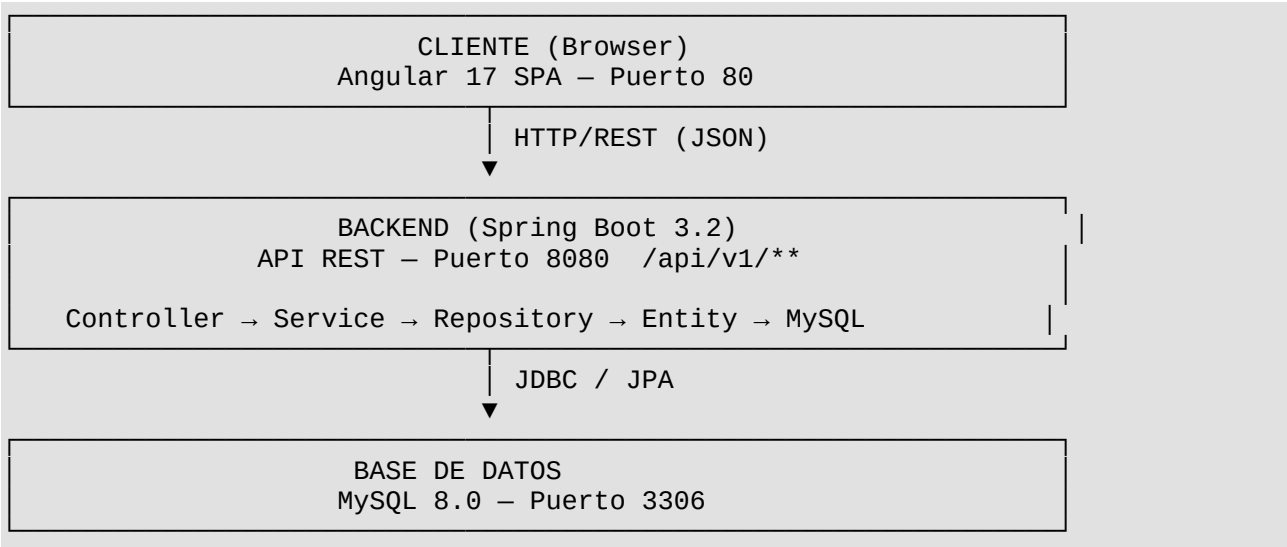
SWO es un sistema web de Service Desk para la gestión integral de incidencias tecnológicas. Permite a los usuarios reportar problemas, hacer seguimiento del estado de sus tickets, administrar proyectos y gestionar el equipo técnico, con soporte de un chatbot de asistencia.

Características principales:

- Registro y seguimiento de incidencias (tickets)
- Gestión de proyectos y asignación de equipos
- Administración de usuarios y roles
- Reportes y estadísticas en tiempo real
- Chatbot de soporte automatizado
- Dashboard ejecutivo con métricas clave

Arquitectura

El sistema sigue una arquitectura de **tres capas** con separación clara entre frontend, backend y base de datos:



Patrones de diseño aplicados:

- Repository Pattern (acceso a datos)
- Dependency Injection (Spring / Angular)
- DTO Pattern (transferencia de datos)
- Strategy Pattern (servicios intercambiables)
- Observer Pattern (RxJS en Angular)
- Global Exception Handler
- Smart / Dumb Components (Angular)

Módulos del Sistema

Módulo	Descripción	Frontend	Backend
Auth	Autenticación y autorización JWT	features/auth	SecurityConfig
Dashboard	Panel de control con métricas	features/dashboard	/v1/estadisticas
Incidencias	CRUD completo de tickets	features/incidents	/v1/incidencias
Proyectos	Gestión de proyectos y asignaciones	features/projects	/v1/proyectos
Usuarios	Administración de	features/users	/v1/usuarios

Módulo	Descripción	Frontend	Backend
	usuarios y roles		
Categorías	Clasificación de incidencias	Integrado	/v1/categorias
Chatbot	Asistente virtual de soporte	features/ chatbot	/v1/chatbot
Reportes	Generación de reportes PDF/Excel	features/ reports	/v1/reportes

Stack Tecnológico

Frontend

Tecnología	Versión	Propósito
Angular	17.0+	Framework SPA
TypeScript	5.2	Lenguaje de programación
RxJS	7.8	Programación reactiva
jsPDF	4.2	Generación de PDFs
Jasmine/Karma	5.1/6.4	Pruebas unitarias
Cypress	13.6	Pruebas E2E

Backend

Tecnología	Versión	Propósito
Java	17 LTS	Lenguaje de programación
Spring Boot	3.2.4	Framework principal
Spring Data JPA	3.2.4	ORM y acceso a datos
Spring Security	3.2.4	Autenticación/Autorización
Springdoc OpenAPI	2.3.0	Documentación API Swagger
Lombok	1.18.32	Reducción de código boilerplate
MapStruct	1.5.5	Mapeo de entidades/DTOs
JUnit 5	5.10	Framework de pruebas
Mockito	5.3.1	Mocks para pruebas unitarias
JaCoCo	0.8.10	Cobertura de código (min. 80%)

Base de Datos

Tecnología	Versión	Propósito
MySQL	8.0.35	Base de datos relacional
phpMyAdmin	5.2	Administración visual BD

Infraestructura

Tecnología	Versión	Propósito
Docker	24+	Contenedorización
Docker Compose	3.9	Orquestación de servicios
Nginx	Latest	Servidor web / Reverse Proxy
Maven	3.8+	Gestión de dependencias Java
npm	9+	Gestión de dependencias JS

Requisitos Previos

Opción A — Con Docker (Recomendado)

- Docker Desktop 24+
- Docker Compose 3.9+

Opción B — Sin Docker

- Java 17 LTS (OpenJDK o Eclipse Temurin)
- Node.js 18+ y npm 9+
- MySQL 8.0+
- Maven 3.8+
- Angular CLI 17+: `npm install -g @angular/cli@17`

Instalación y Ejecución

Con Docker Compose

```
# 1. Clonar el repositorio
git clone https://github.com/JuanPGE123/SW0.git
cd SW0

# 2. Configurar variables de entorno
cp .env.example .env
# Editar .env con los valores apropiados
```

```
# 3. Iniciar todos los servicios
docker-compose up -d

# 4. Verificar que los servicios están corriendo
docker-compose ps

# 5. Ver logs en tiempo real
docker-compose logs -f backend
```

URLs disponibles con Docker: | Servicio | URL | |-----|-----| | Frontend (Angular) |
http://localhost:80 | | Backend (API REST) | http://localhost:8080/api | | Swagger UI |
http://localhost:8080/api/swagger-ui.html | | phpMyAdmin | http://localhost:8888 (perfil dev) |

Sin Docker — Desarrollo Local

Backend (Spring Boot)

```
cd backend

# Opción 1: Maven Wrapper
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev

# Opción 2: Maven instalado
mvn spring-boot:run -Dspring.profiles.active=dev

# Opción 3: JAR compilado
mvn clean package -DskipTests
java -jar target/swo-api-1.0.0.jar --spring.profiles.active=dev
```

Frontend (Angular)

```
# Desde la raíz del proyecto
npm install
npm start
# Disponible en: http://localhost:4200
```

Windows — Script de inicio completo

```
INICIAR_TODO.bat
```

Variables de Entorno

Copiar `.env.example` como `.env` y configurar:

Variable	Descripción	Ejemplo
DB_HOST	Host de MySQL	localhost
DB_PORT	Puerto MySQL	3306
DB_NAME	Nombre de la BD	swo_dev
DB_USER	Usuario BD	root
DB_PASSWORD	Contraseña BD	root

Variable	Descripción	Ejemplo
JWT_SECRET_KEY	Clave secreta JWT	clave_muy_segura...
JWT_EXPIRATION_TIME	Expiración token (ms)	86400000
SPRING_PROFILES_ACTIVE	Perfil activo	dev / test / prod
NG_APP_API_URL	URL del backend	http://localhost:8080/api

NUNCA commitear el archivo `.env`. Está incluido en `.gitignore`.

API REST — Endpoints

La documentación interactiva completa está disponible en Swagger UI:

<http://localhost:8080/api/swagger-ui.html>

Resumen de Endpoints

Módulo	Método	Endpoint	Descripción
Incidencias	GET	/v1/incidencias	Listar todas (paginado)
	GET	/v1/incidencias/{id}	Obtener por ID
	GET	/v1/incidencias/estado/{estado}	Filtrar por estado
	GET	/v1/incidencias/usuario/{id}	Por usuario
	GET	/v1/incidencias/buscar?q=texto	Búsqueda libre
	POST	/v1/incidencias	Crear incidencia
	PUT	/v1/incidencias/{id}	Actualizar incidencia
	PATCH	/v1/incidencias/{id}/estado	Cambiar estado
	DELETE	/v1/incidencias/{id}	Eliminar incidencia
Proyectos	GET	/v1/proyectos	Listar proyectos

Módulo	Método	Endpoint	Descripción
	GET	/v1/proyectos/{id}	Obtener proyecto
	POST	/v1/proyectos	Crear proyecto
	PUT	/v1/proyectos/{id}	Actualizar proyecto
	DELETE	/v1/proyectos/{id}	Eliminar proyecto
Usuarios	GET	/v1/usuarios	Listar usuarios
	GET	/v1/usuarios/{id}	Obtener usuario
	POST	/v1/usuarios	Crear usuario
	PUT	/v1/usuarios/{id}	Actualizar usuario
Categorías	GET	/v1/categorias	Listar categorías
	POST	/v1/categorias	Crear categoría
Chatbot	POST	/v1/chatbot/mensaje	Enviar mensaje
	GET	/v1/chatbot/historial/{id}	Historial de conversación

Formato de Respuesta

Todas las respuestas siguen el formato estándar `ApiResponse<T>`:

```
{
  "success": true,
  "message": "Operación exitosa",
  "data": { ... },
  "timestamp": "2026-06-02T10:00:00Z"
}
```

Códigos HTTP utilizados:

Código	Significado
200 OK	Operación exitosa
201 Created	Recurso creado
204 No Content	Eliminación exitosa
400 Bad Request	Datos de entrada inválidos
401 Unauthorized	No autenticado
403 Forbidden	Sin permisos
404 Not Found	Recurso no encontrado
500 Internal Server Error	Error del servidor

Pruebas

Pruebas Unitarias — Backend

```
cd backend

# Ejecutar todas las pruebas
mvn test

# Con reporte de cobertura JaCoCo (mín. 80%)
mvn test jacoco:report

# Ver reporte en: backend/target/site/jacoco/index.html
```

Pruebas Unitarias — Frontend

```
# Desde la raíz del proyecto

# Modo interactivo (con Chrome)
npm test

# Modo CI (Chrome headless + cobertura)
npm run test:coverage
```

Pruebas E2E — Cypress

```
# Requiere que frontend y backend estén corriendo

# Modo interactivo
npx cypress open

# Modo headless (CI)
npm run e2e:headless
```

Ver resultados detallados en [INFORME_PRUEBAS.md](#).

Despliegue

Ambientes Disponibles

Ambiente	Descripción	Configuración
Desarrollo	Local del desarrollador	.env + SPRING_PROFILES_ACTIV E=dev
Pruebas	Pruebas automatizadas CI	BD H2 en memoria + profile=test
Producción	Contenedores Docker	docker-compose.yml + profile=prod

Despliegue con Docker (Producción)

```
# Construir imágenes
docker-compose build

# Iniciar en modo producción
docker-compose --env-file .env up -d

# Escalar backend (múltiples instancias)
docker-compose up -d --scale backend=3
```

Archivos Ejecutables

Artefacto	Ruta	Descripción
Backend JAR	backend/target/swo-api-1.0.0.jar	Ejecutable Spring Boot
Frontend dist	dist/swo-servicedesk/	Build de producción Angular
Docker Image	docker build -t swo-app .	Imagen multi-stage

Generar Artefactos

```
# Backend – genera JAR ejecutable
cd backend && mvn clean package -DskipTests

# Frontend – genera build de producción
npm run build:prod

# Script de build completo (Windows)
build.bat

# Script de build completo (Linux/Mac)
./build.sh
```

Control de Versiones

Repositorio: <https://github.com/JuanPGE123/SWO>

Convención de Commits (Conventional Commits)

```
feat: Nueva funcionalidad
fix: Corrección de error
docs: Cambios en documentación
style: Formato (no afecta lógica)
refactor: Refactorización de código
test: Agregar o actualizar pruebas
chore: Tareas de mantenimiento
```

Ramas

Rama	Propósito
main	Código estable — producción

Rama	Propósito
develop	Integración de features
feature/*	Nuevas funcionalidades
fix/*	Correcciones de errores
release/*	Preparación de versiones

Estructura del Proyecto

SWO/	
├── backend/	# Spring Boot API REST
│ └── src/	
│ ├── main/java/com/swo/api/	
│ ├── controller/	# Controladores REST
│ ├── service/	# Lógica de negocio
│ └── impl/	# Implementaciones
│ ├── repository/	# Acceso a datos (JPA)
│ ├── model/	
│ ├── entity/	# Entidades JPA
│ ├── dto/	
│ ├── request/	# DTOs de entrada
│ └── response/	# DTOs de salida
│ └── enums/	# Enumeraciones
│ ├── config/	# Configuraciones Spring
│ ├── exception/	# Manejo de excepciones
│ ├── common/	# Utilitarios comunes
│ └── test/	# Pruebas unitarias
├── src/	# Angular 17 Frontend
│ └── app/	
│ ├── core/	# Servicios, guards, interceptores
│ ├── shared/	# Componentes reutilizables
│ └── features/	# Módulos funcionales
│ ├── auth/	
│ ├── dashboard/	
│ ├── incidents/	
│ ├── projects/	
│ ├── users/	
│ ├── chatbot/	
│ └── reports/	
├── cypress/	# Pruebas E2E
├── java/	# Scripts SQL de BD
├── dist/	# Build frontend (generado)
├── Dockerfile	# Imagen multi-stage
├── docker-compose.yml	# Orquestación de contenedores
├── .env.example	# Plantilla de variables de entorno
├── build.bat / build.sh	# Scripts de build
├── MANUAL_TECNICO.md	# Manual técnico
├── INFORME_PRUEBAS.md	# Resultados de pruebas
└── Documento_Arquitectura_SWO.md	# Documento de arquitectura

Documentación Adicional

Documento	Descripción
MANUAL TECNICO.md	Configuración de servidores, BD y ambientes
INFORME PRUEBAS.md	Resultados de pruebas unitarias, integración y E2E
Documento Arquitectura SWO.md	Arquitectura completa, patrones y decisiones de diseño
Swagger UI	Documentación interactiva de la API REST

Autores

Nombre	Rol
Juan Pablo Giraldo	Desarrollador Full Stack

Institución: SENA — Servicio Nacional de Aprendizaje

Programa: Tecnología en Análisis y Desarrollo de Software

Año: 2026